

Nome do Software: AppScholar

Nome do(a) Aluno(a): Andressa Stéphanie Toledo da Silva

1. Introdução

1.1. Objetivo do Projeto

O objetivo do projeto é desenvolver um aplicativo mobile funcional e multiplataforma de gerenciamento de boletim acadêmico voltado para instituições de graduação em tecnologia. O sistema visa integrar uma interface mobile intuitiva e responsiva a um ecossistema de backend robusto, permitindo a autenticação segura de usuários, o cadastro de informações institucionais (alunos, professores e disciplinas) e a consulta automatizada de notas e rendimentos escolares.

1.2. Escopo do Sistema

O sistema mobile é um aplicativo desenvolvido para Android, iOS ou ambos, com foco em gerenciar e centralizar as informações acadêmicas, realizar cadastros de agentes institucionais, integrar APIs externas de localização para automatização de formulários e disponibilizar a consulta em tempo real do boletim de notas, médias e situações dos estudantes.

1.3. Público-Alvo

Alunos da Graduação em Tecnologia: Usuários finais que utilizarão o aplicativo para consultar suas notas, médias e situação nas disciplinas contratadas.

Secretaria / Administradores Escolares: Responsáveis por alimentar o sistema através das telas de cadastro de alunos, professores e disciplinas.

Professores: Responsáveis por alimentar o sistema com notas para calcular média dos alunos.

1.4. Siglas e/ou Abreviações

- API: Interface de Programação de Aplicações
- CRUD: Create, Read, Update, Delete
- UI/UX: Interface/Experiência do Usuário
- POST
- GET
- useState
- HTTP

2. Visão Geral do Sistema

2.1. Funcionalidades Principais

Cadastro e autenticação de usuários:

- Autenticação Segura: Sistema de login para validação de credenciais (e-mail/login e senha) integrando a interface mobile à API restrita do servidor.
- Gerenciamento de Sessão: Geração e fornecimento de token de autenticação (padrão JWT recomendado) pelo backend no ato do login para controle de acesso seguro e identificação do perfil do usuário (ex: aluno).
- Tratamento de Exceções: Validação de campos obrigatórios do formulário de login diretamente na interface, emitindo feedbacks visuais e alertas caso os campos estejam em branco.

Navegação por abas ou menus:

- Navegação Intuitiva: Arquitetura de navegação nativa estruturada com o pacote React Navigation, garantindo fluidez e legibilidade no ecossistema mobile.
- Painel de Controle Centralizado (Dashboard): Tela inicial que atua como hub principal de navegação, roteando o usuário de maneira simplificada por meio de cartões (cards), botões ou menus para as seções de cadastro e consultas acadêmicas.

Integração com banco de dados remoto:

- Persistência Relacional: Backend em Node.js e Express.js conectado diretamente a uma base de dados relacional PostgreSQL.
- Gravação de Registros Acadêmicos: Endpoints específicos baseados no protocolo HTTP (POST) mapeados para receber e estruturar fluxos de dados enviados pelo app mobile, salvando-os nas tabelas correspondentes de alunos, professores e disciplinas.
- Sincronização e Leitura Relacional: Consolidação e cruzamento de dados via chaves estrangeiras entre as tabelas do banco de dados para montagem e entrega dinâmica da resposta do Boletim de Notas do estudante através de requisições de consulta (GET).

Acesso offline limitado ou total:

- Acesso Offline Limitado: Por exigir conexões ativas para autenticação em tempo real por JWT, consultas ao banco de dados relacional e requisições dinâmicas a APIs de terceiros, o sistema opera de forma estritamente conectada. No entanto, a aplicação faz uso de gerenciamento de estado local via Hooks (useState) para retenção temporária e controle de dados manipulados nas telas de formulários antes do envio definitivo para o servidor.

Funcionalidades específicas:

- Automatização de Endereço Residencial (ViaCEP): Integração automática do formulário mobile de Cadastro de Alunos com a API pública do ViaCEP. Ao digitar um CEP válido, o sistema dispara uma requisição HTTP e preenche previamente os campos de endereço, cidade e estado.

- Localidades Dinâmicas (IBGE): Consumo assíncrono dos serviços de dados do IBGE para carregar de forma nativa e padronizada as listas de estados e municípios nos componentes de seleção dos formulários acadêmicos.
- Cálculo e Emissão de Rendimento Escolar: Motor interno de regras no banco/backend encarregado de computar as notas parciais (nota1 e nota2), gerar a média aritmética e injetar automaticamente a string de situação acadêmica ("Aprovado" ou "Reprovado") para visualização no Boletim do estudante.

2.2. Plataformas Suportadas

- Android
- iOS

3. Requisitos do Sistema

3.1. Requisitos Funcionais

- RF001: O sistema deve permitir a autenticação de usuários por e-mail/login e senha.
- RF002: O sistema deve gerar e gerenciar um token JWT após o login bem-sucedido para controle de sessão.
- RF003: O sistema deve validar o preenchimento de campos obrigatórios do formulário de login antes do envio.
- RF004: O sistema deve exibir uma Dashboard principal com opções de navegação direta para os módulos de cadastros e boletim.
- RF005: O sistema deve permitir o cadastro de dados de novos alunos no banco de dados.
- RF006: O sistema deve permitir o cadastro de dados de novos professores no banco de dados.
- RF007: O sistema deve permitir o cadastro de dados de novas disciplinas vinculadas aos seus respectivos professores responsáveis.
- RF008: O sistema deve consultar de forma dinâmica o boletim acadêmico por meio da matrícula do aluno.
- RF009: O sistema deve realizar a integração com a API ViaCEP para o preenchimento automático dos campos de endereço, cidade e estado no cadastro de alunos através do CEP informado.
- RF010: O sistema deve integrar-se com a API do IBGE Localidades para carregar as listas dinâmicas de estados e cidades brasileiras nos cadastros.
- RF011: O sistema deve processar o cálculo da média aritmética com base nas notas parciais fornecidas e definir automaticamente a situação acadêmica do estudante.

3.2. Requisitos Não Funcionais

- RNF001 (Portabilidade): O aplicativo mobile deve ser multiplataforma, operando nativamente em sistemas operacionais Android e iOS.
- RNF002 (Usabilidade/Design): A interface deve seguir princípios de UI/UX Mobile, apresentando um layout limpo, boa legibilidade e elementos visuais padronizados.
- RNF003 (Feedback ao Usuário): O aplicativo mobile deve fornecer respostas visuais imediatas para as ações do usuário nas telas, incluindo mensagens de erro em validações de formulário.
- RNF004 (Modularidade do Código): O código do aplicativo deve ser desenvolvido utilizando componentes reutilizáveis, hooks customizados e separação de serviços de navegação.
- RNF005 (Arquitetura de API): A comunicação entre o frontend mobile e o backend deve ser estabelecida sob o padrão arquitetural REST, trocando dados estruturados no formato JSON.
- RNF006 (Estrutura de Backend): O backend deve ser estruturado em camadas lógicas distintas utilizando a arquitetura de controllers, routes, models e database em ambiente Node.js.

4. Arquitetura do Sistema

4.1. Tecnologias Utilizadas

- Linguagem: TypeScript
- Framework: React Native (com Expo)
- Banco de Dados: Supabase
- Back-end: Node.js
- API: RESTful

4.2. Camadas da Arquitetura

- Apresentação (Front-end): Interface com o usuário
- Serviço: Requisições HTTP para o back-end
- Negócio: Validações e regras de negócio locais
- Persistência: Armazenamento local e sincronização com servidor

5. Modelagem de Dados

5.1. Diagrama de Entidade-Relacionamento (DER)

Curso: Registra as informações dos cursos (cursoId, cursoNome, cursoPeriodo, cursoMediaAprovacao, cursoDuracao).

Aluno: Registra os dados pessoais, acadêmicos e o endereço do estudante (alunoId, alunoMatricula, alunoNome, alunoEmail, alunoTelefone, alunoCep, alunoEndereco, alunoCidade, alunoEstado, alunoSemestreAtual), apontando para um Curso (cursoId).

Professor: Registra o corpo docente (professorId, professorNome, professorTitulacao, professorAreaAtuacao, professorTempoDocencia, professorEmail).

Disciplina: Registra as matérias da instituição (disciplinaId, disciplinaNome, disciplinaCargaHoraria, disciplinaSemestre, disciplinaAtiva), ligadas a um Professor (professorId) e a um Curso (cursoId).

Boletim: Tabela associativa que guarda o histórico de notas e a situação final do aluno numa determinada matéria (boletimId, alunoId, disciplinaId, boletimNota1, boletimNota2, boletimMedia, boletimSituacao).

Usuário: Centraliza o acesso e a segurança do sistema (usuarioId, usuarioNome, usuarioEmail, usuarioSenha, usuarioRole, usuarioAtivo), vinculando-se opcionalmente a um alunoId ou professorId.

Vínculo de Cursos com Alunos e Disciplinas: A tabela de **Curso** funciona como a base organizacional da instituição. Cada curso criado pode possuir vários alunos matriculados e, da mesma forma, é composto por diversas disciplinas em sua grade curricular. No entanto, para manter a consistência, cada **Aluno** e cada **Disciplina** pertencem exclusivamente a um único curso.

Atribuição de Professores às Disciplinas: Cada **Disciplina** cadastrada no sistema deve ter, obrigatoriamente, um **Professor** responsável por ministrá-la. Por outro lado, a estrutura permite flexibilidade para o corpo docente, o que significa que um mesmo professor pode ser vinculado à regência de várias disciplinas ao mesmo tempo.

A Composição do Boletim (Rendimento Acadêmico): O **Boletim** funciona como uma tabela de ligação fundamental que une o histórico do aluno ao conteúdo estudado. Ele cruza os dados para registrar o desempenho escolar: um **Aluno** acumula múltiplos boletins ao longo do tempo (um para cada matéria que ele cursa), e cada **Disciplina** gera vários registros de boletins para relacionar as notas de todos os estudantes que estão matriculados nela. É nesta conexão que as notas 1 e 2 são guardadas e calculadas.

Controle de Acesso e Perfis de Usuários: Para garantir a segurança, a tabela de **Usuario** centraliza as credenciais de login (e-mail e senha) e define o nível de permissão (administrador, professor ou aluno). Esse controle é feito por vínculos individuais e opcionais: quando a conta pertence a um estudante, o usuário se conecta diretamente a um registro da tabela **Aluno**; quando pertence a um docente, conecta-se a um registro da tabela **Professor**. Isso impede que um aluno acesse telas de lançamento de notas ou que dados fiquem sem um responsável autenticado.

5.2. Estrutura de Tabelas (exemplo)

Tabela: Curso

- **cursoId:** texto (UUID), chave primária
- **cursoNome:** texto, único
- **cursoPeriodo:** texto (Enum: Matutino, Vespertino, Noturno, Integral)
- **cursoMediaAprovacao:** decimal/ponto flutuante, opcional
- **cursoDuracao:** inteiro, opcional

Tabela: Aluno

- **alunoId:** texto (UUID), chave primária
- **alunoNome:** texto
- **alunoMatricula:** texto, único
- **alunoEmail:** texto, único
- **alunoTelefone:** texto
- **alunoCep:** texto
- **alunoEndereco:** texto
- **alunoCidade:** texto
- **alunoEstado:** texto
- **alunoSemestreAtual:** inteiro, padrão: 1
- **cursoId:** texto (UUID), chave estrangeira (referencia Curso)
- **alunoCreatedAt:** data e hora, padrão: data atual

Tabela: Professor

- **professorId:** texto (UUID), chave primária
- **professorNome:** texto
- **professorTitulacao:** texto
- **professorAreaAtuacao:** texto
- **professorTempoDocencia:** inteiro
- **professorEmail:** texto, único

Tabela: Disciplina

- disciplinaId: texto (UUID), chave primária
- disciplinaNome: texto
- disciplinaCargaHoraria: inteiro
- disciplinaSemestre: inteiro
- disciplinaAtiva: booleano, padrão: verdadeiro
- professorId: texto (UUID), chave estrangeira (referencia Professor)
- cursold: texto (UUID), chave estrangeira (referencia Curso)

Tabela: Boletim

- boletimId: texto (UUID), chave primária
- alunold: texto (UUID), chave estrangeira (referencia Aluno)
- disciplinaId: texto (UUID), chave estrangeira (referencia Disciplina)
- boletimNota1: decimal/ponto flutuante
- boletimNota2: decimal/ponto flutuante
- boletimMedia: decimal/ponto flutuante
- boletimSituacao: texto (Enum: NaoCursado, EmAndamento, Aprovado, Reprovado, EmRecuperacao, Trancado)
- boletimCreatedAt: data e hora, padrão: data atual

Tabela: Usuario

- usuarioId: texto (UUID), chave primária
- usuarioNome: texto
- usuarioEmail: texto, único
- usuarioSenha: texto criptografado
- usuarioRole: texto (Enum: admin, professor, aluno), padrão: aluno
- usuarioAtivo: booleano, padrão: verdadeiro
- alunold: texto (UUID), chave estrangeira opcional, única (referencia Aluno)
- professorId: texto (UUID), chave estrangeira opcional, única (referencia Professor)
- usuarioCreatedAt: data e hora, padrão: data atual
- usuarioUpdatedAt: data e hora, atualizado automaticamente

6. Fluxos de Navegação

6.1. Telas e Componentes

- Tela de login
- Tela de home
- Tela de aluno
- Tela de curso
- Tela de disciplina
- Tela de boletim
- Tela de professor
- Telas de CRUD de cada uma das telas anteriores

6.2. Diagrama de Navegação

Fluxo de Autenticação (Inicial):

- O usuário abre o aplicativo e é direcionado obrigatoriamente para a **Tela de Login**.
- Se as credenciais forem válidas e o token JWT for gerado, o sistema realiza a transição de rotas e abre a **Tela de Home**.

Fluxo Principal (A partir da Home):

- A **Tela de Home** atua como o nó central de navegação do aplicativo. A partir dela, o usuário tem canais diretos de redirecionamento para:
 - **Tela de Perfil / Mudar Senha:** Onde o usuário gerencia a segurança da sua conta.
 - **Módulos de Consulta e CRUDs:** Menus ou cards que abrem as telas de listagem e gerenciamento de **Alunos, Cursos, Disciplinas, Professores e Boletins**.
- Todas as telas secundárias possuem um botão nativo de retorno no cabeçalho, permitindo que o usuário regresse facilmente à **Tela de Home**.

7. Segurança

7.1. Autenticação e Autorização

- Uso de JWT (JSON Web Token)
- Senhas criptografadas (bcrypt ou outra biblioteca)

7.2. Armazenamento Seguro

- Uso de AsyncStorage com criptografia ou Keychain

8. Testes

8.1. Tipos de Testes Aplicados

- Testes Unitários
- Testes de Integração
- Testes de Usabilidade

8.2. Ferramenta de Teste (Sugerida)

Não se aplica

9. Implantação

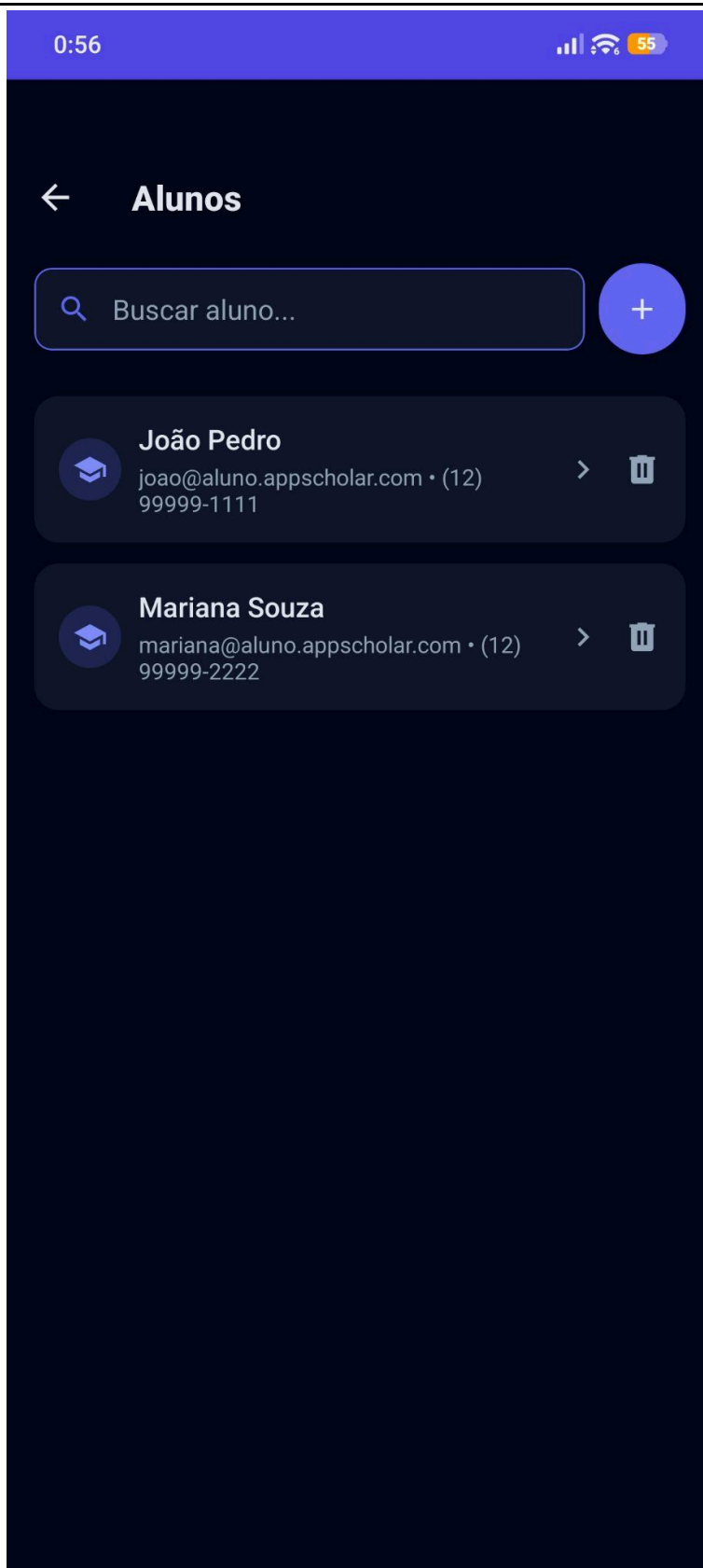
Não se aplica

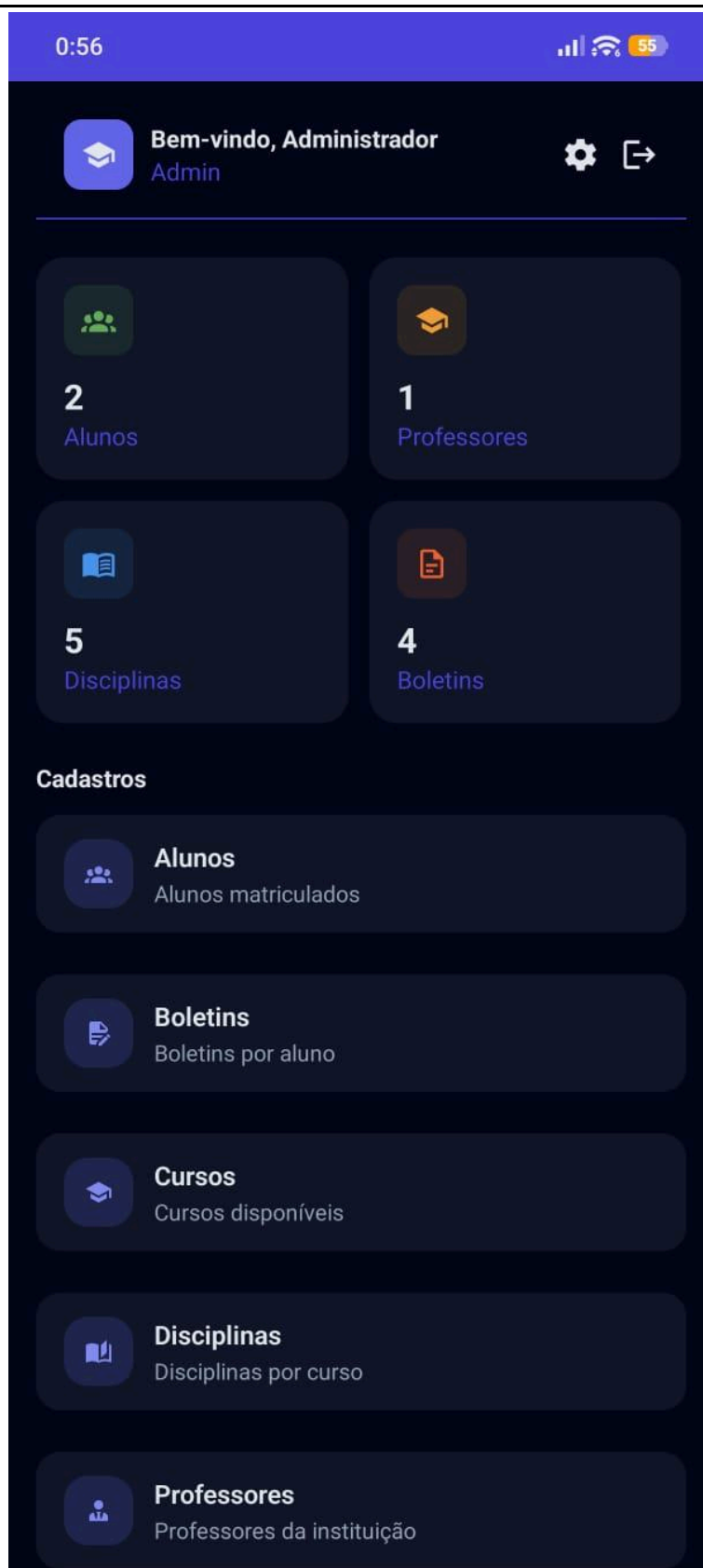
10. Manutenção e Atualizações

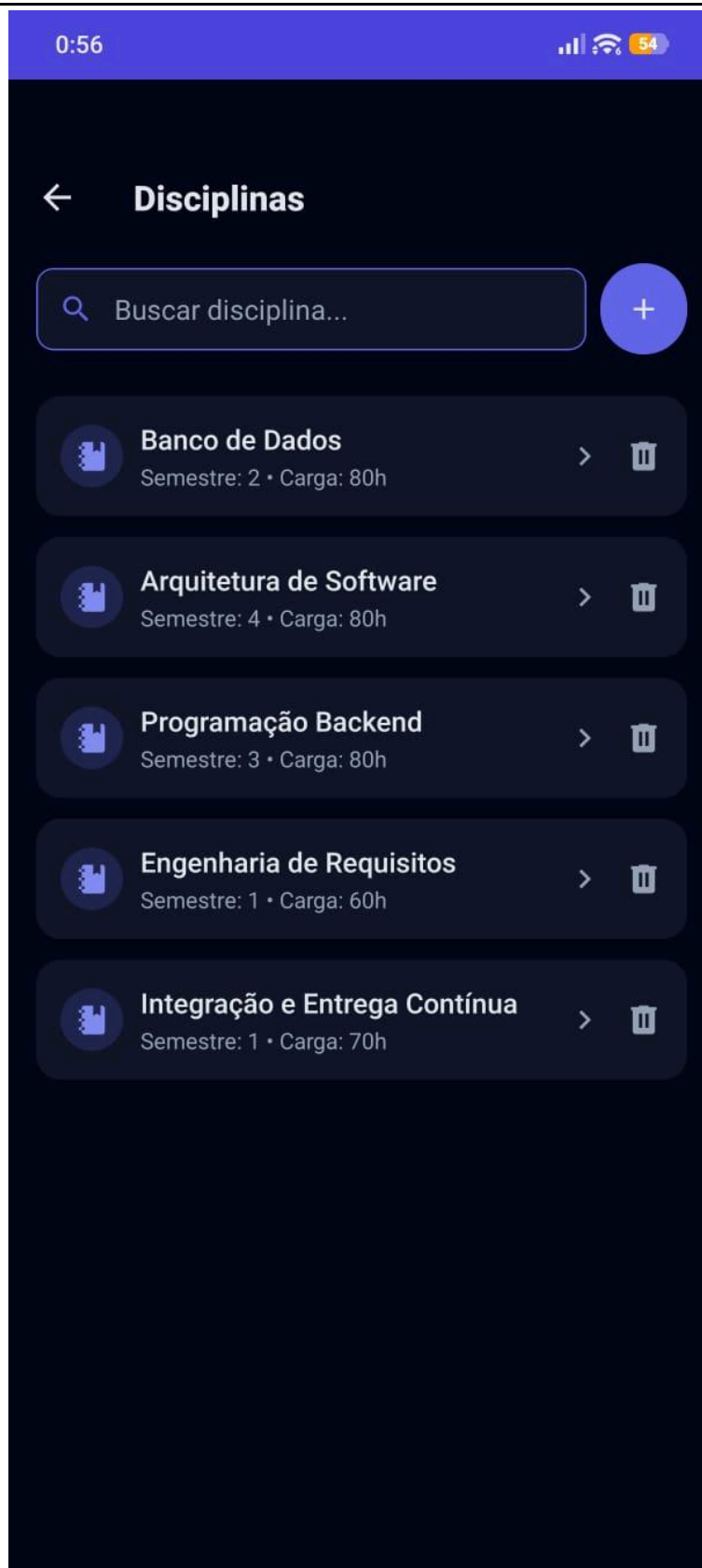
Não se aplica

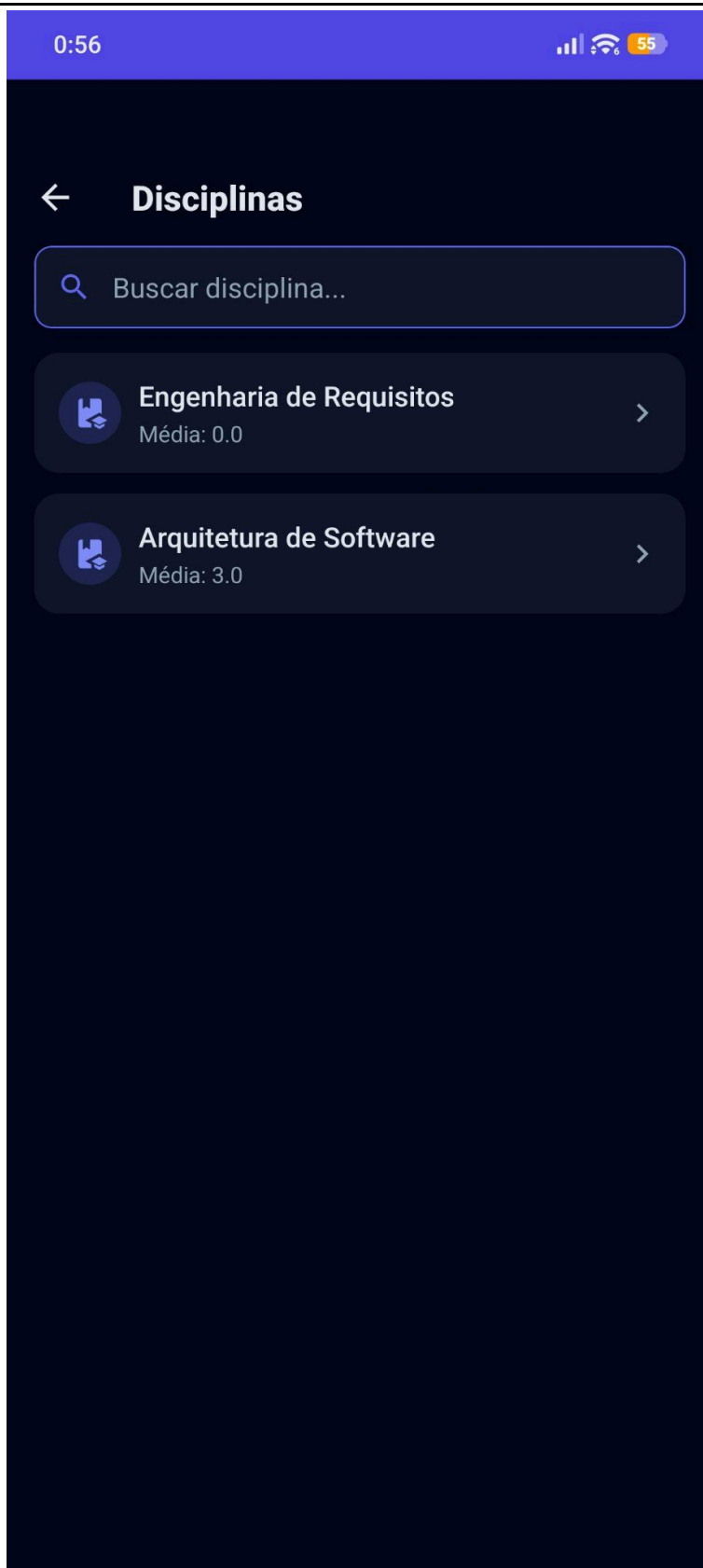
11. Anexos











0:56   55

 **Alunos**

Nome completo *

João Pedro

Matrícula *

2026001

Curso *

Engenharia de Software 

E-mail *

joao@aluno.appscholar.com

Telefone *

(12) 99999-1111

CEP *

76824-512

Endereço *

Rua Roberto de Souza

Estado *

0:56    54

 **Disciplinas**

Nome da disciplina *

Arquitetura de Software

Carga horária *

80

Semestre *

4

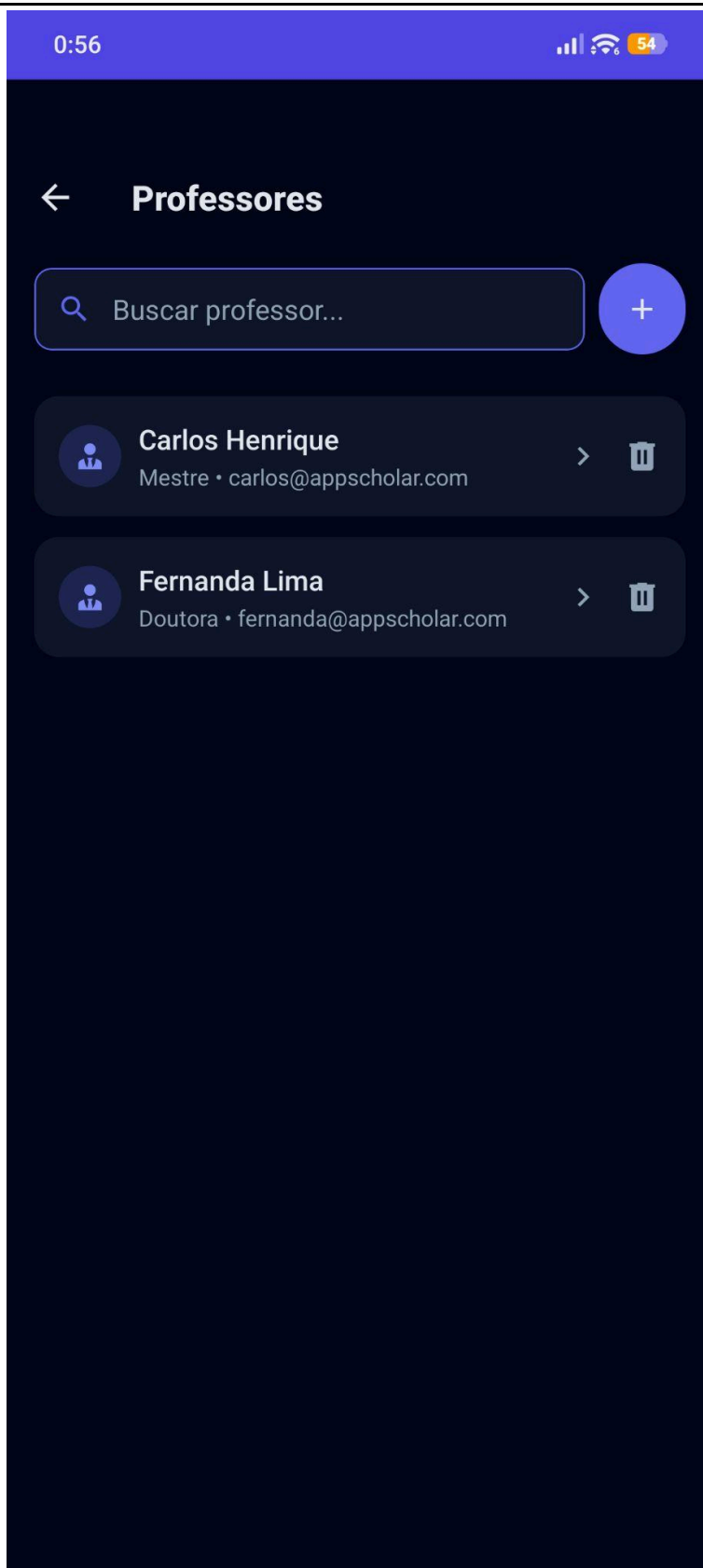
Professor *

Fernanda Lima 

Curso *

Engenharia de Software 

Atualizar



0:56   

 **Professores**

Nome completo *

Fernanda Lima

Titulação *

Doutora

Área de atuação *

Banco de Dados

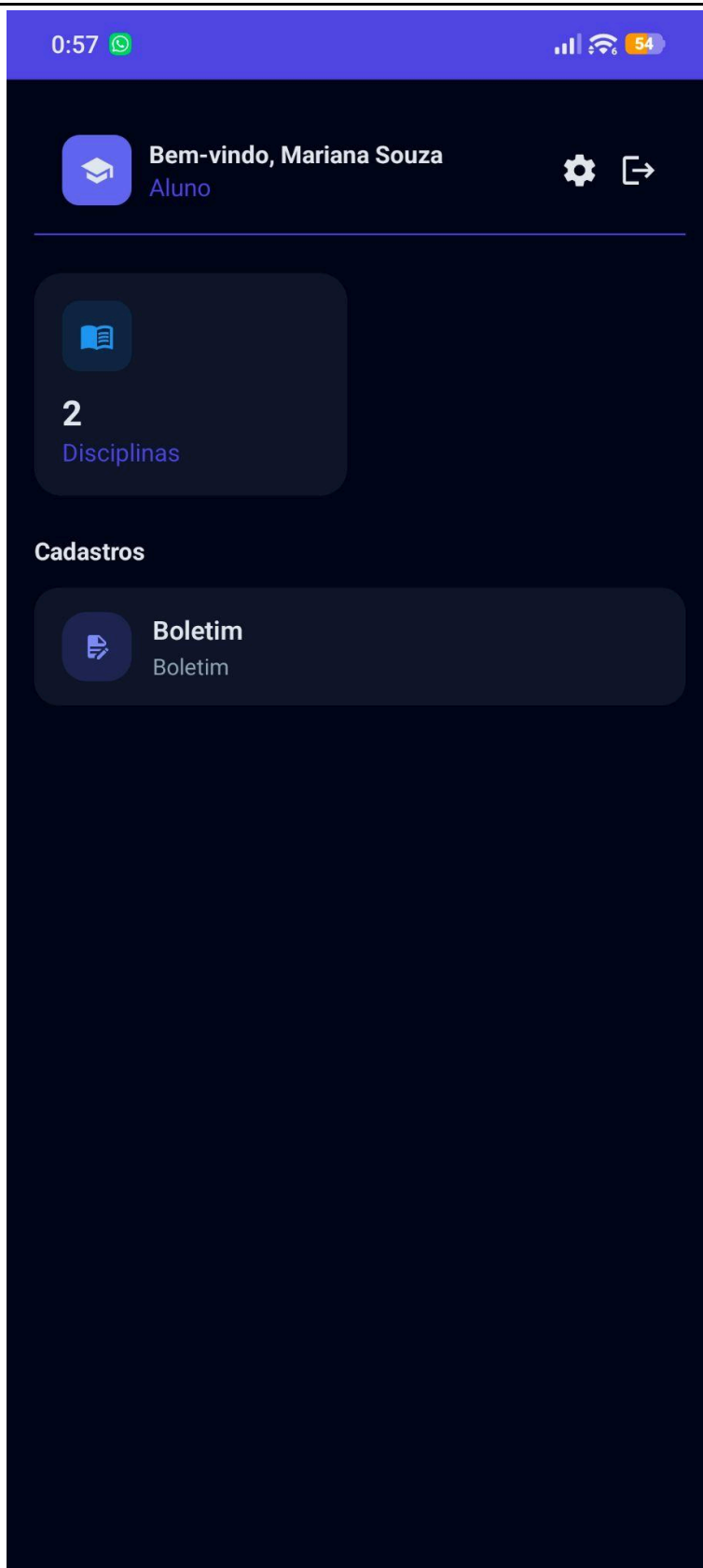
Tempo de docência *

15

E-mail *

fernanda@appscholar.com

Atualizar



0:57   

 **Boletim**

Engenharia de Requisitos

Nota 1

0

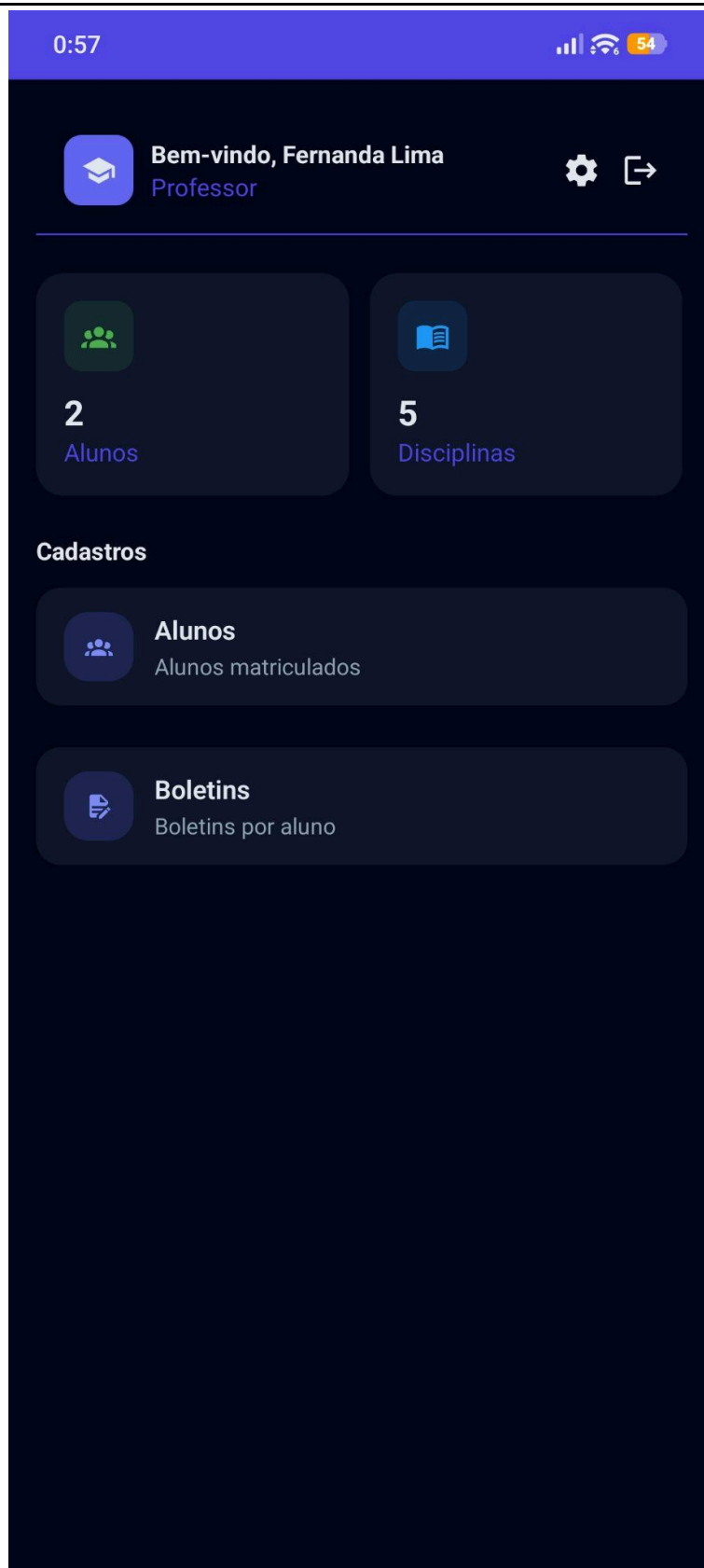
Nota 2

0

Média

0.0

Atualizar notas



0:57

<
Boletim

Engenharia de Software
Semestre Atual: 1º

Aprovadas
0

Reprovadas
0

Andamento
1

Pendentes
0

Todos

1º

Buscar disciplina...

Engenharia de Requisitos

Em andamento

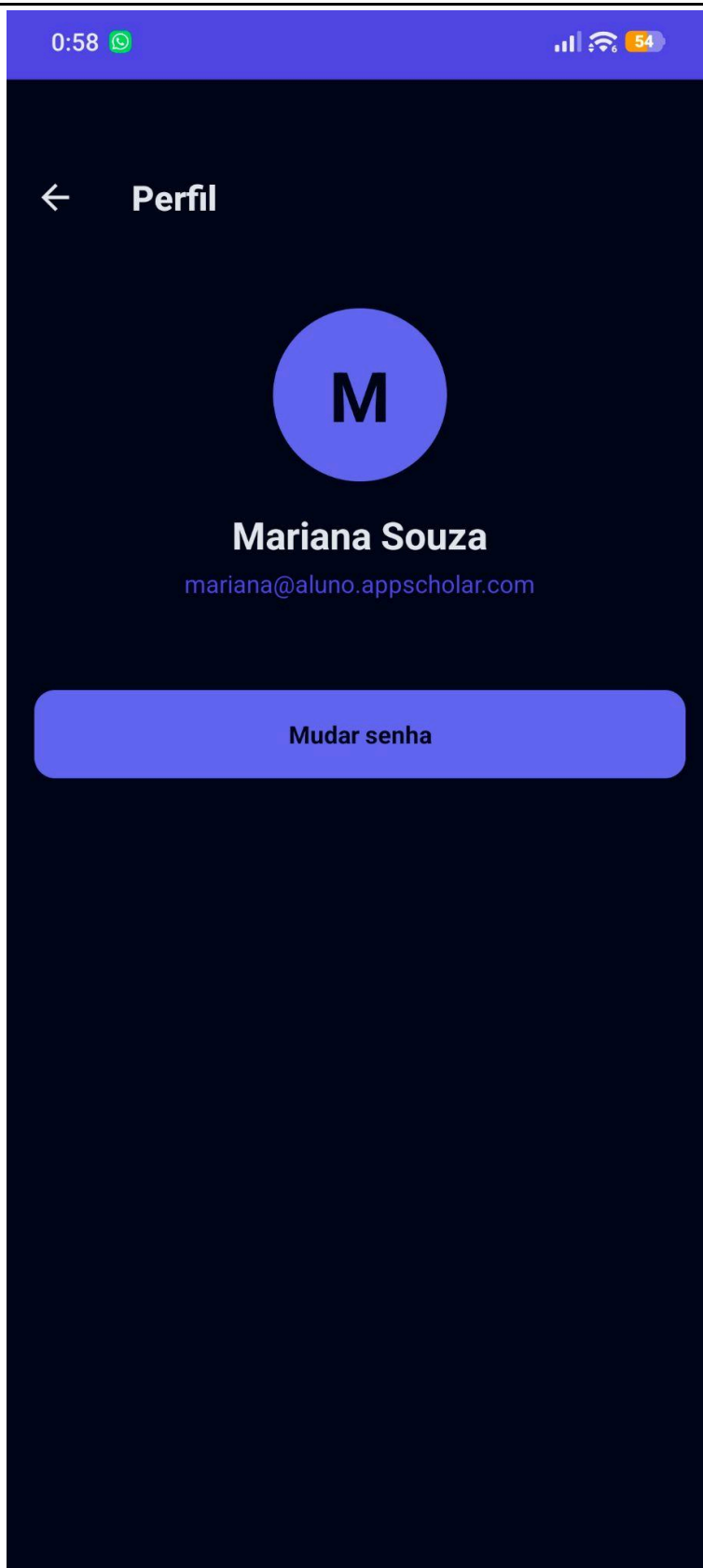
Professor	Semestre	Carga Horária
Fernanda Lima	1º	60h

0.0
NOTA 1

0.0
NOTA 2

0.0
MÉDIA

Desempenho
0%



0:58     54

← **Mudar Senha**

Senha atual

Nova senha

Confirmar senha

Salvar nova senha